

1877. Minimize Maximum Pair Sum in Array

Solved 

Medium

 Topics

 Companies

 Hint

The **pair sum** of a pair (a, b) is equal to $a + b$. The **maximum pair sum** is the largest **pair sum** in a list of pairs.

- For example, if we have pairs $(1, 5)$, $(2, 3)$, and $(4, 4)$, the **maximum pair sum** would be $\max(1+5, 2+3, 4+4) = \max(6, 5, 8) = 8$.

Given an array `nums` of **even** length `n`, pair up the elements of `nums` into $n / 2$ pairs such that:

- Each element of `nums` is in **exactly one** pair, and
- The **maximum pair sum** is **minimized**.

Return the *minimized maximum pair sum* after optimally pairing up the elements.

Example 1:

Input: `nums = [3,5,2,3]`

Output: 7

Explanation: The elements can be paired up into pairs $(3, 3)$ and $(5, 2)$.
The maximum pair sum is $\max(3+3, 5+2) = \max(6, 7) = 7$.

Example 2:

Input: `nums = [3,5,4,2,4,6]`

Output: 8

Explanation: The elements can be paired up into pairs $(3, 5)$, $(4, 4)$, and $(6, 2)$.
The maximum pair sum is $\max(3+5, 4+4, 6+2) = \max(8, 8, 8) = 8$.

Constraints:

- `n == nums.length`
- `2 <= n <= 105`
- `n` is **even**.
- `1 <= nums[i] <= 105`

Python:

class Solution:

```
def minPairSum(self, nums: List[int]) -> int:  
    return nums.sort() or max(nums[i]+nums[-1-i] for i in range((len(nums)>>1)+1))
```

JavaScript:

```
var minPairSum = function(nums) {  
    // Sort the array in ascending order  
    nums.sort((a, b) => a - b);  
  
    let n = nums.length - 1;  
    let maximum = nums[0] + nums[n];  
  
    // Pair elements equidistant from both ends  
    for (let i = 1; i < n; i++) {  
        maximum = Math.max(maximum, nums[i] + nums[n - i]);  
    }  
  
    return maximum;  
};
```

Java:

```
class Solution {  
    public int minPairSum(int[] nums) {  
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;  
        int[] freq = new int[100001];  
        for(int i = 0; i < nums.length; i++) {  
            if(nums[i] < min) min = nums[i];  
            if(nums[i] > max) max = nums[i];  
            freq[nums[i]]++;  
        }  
        int max_sum = 0, l = min, r = max;  
        while(l <= r) {  
            if(freq[l] == 0) l++;  
            else if(freq[r] == 0) r--;  
            else {  
                max_sum = Math.max(max_sum, l + r);  
                freq[l]--;  
                freq[r]--;  
            }  
        }  
        return max_sum; }}
```